

Sistema de Gestión y Resguardo de Activos TI

OBJETIVO

Desarrollar una API para administrar activos de tecnología de una empresa, tales como laptops, monitores, impresoras, celulares, periféricos y equipos arrendados. Con la finalidad de evaluar diseño de APIs, lógica de negocio, SQL SERVER, Stored Procedures, seguridad, middleware, manejo de concurrencia y calidad de código.

La solución deberá permitir conocer:

- Qué activos existen.
- A quién se encuentran asignados.
- En qué ubicación están.
- Qué proveedor los suministró, arrendó o da mantenimiento.
- El historial de movimientos de cada activo.
- Qué activos se encuentran disponibles, asignados, en mantenimiento o dados de baja.

TECNOLOGÍAS REQUERIDAS

Backend (obligatorio):

- ASP.NET Core Web API.
- .NET 8 o superior.
- SQL Server.
- ADO.NET mediante Microsoft.Data.SqlClient.
- Stored Procedures para operaciones principales.
- JWT para autenticación.
- Swagger / OpenAPI.
- Pruebas unitarias o de integración.

Frontend(NO ES OBLIGATORIO): En caso de incluirlo, puede utilizar React, Angular, Vue, Blazor u otra tecnología de su preferencia.

La evaluación principal será sobre backend, seguridad, base de datos y lógica de negocio. El frontend será considerado como valor adicional.

TIEMPO ESPERADO

La prueba está diseñada para una ser entregada 3 día después de recibir este instructivo. Puedes entregar una solución parcial, siempre que documentes claramente las funcionalidades que quedaron pendientes, como continuaría la implementación y que riegos identifico.

No se espera un producto terminado ni un diseño visual complejo.

CONTEXTO DEL PROBLEMA

La empresa requiere administrar activos de TI. Los activos pueden ser propios o arrendados dependiendo del proveedor, el sistema también debe administrar el tipo de servicios que brinda cada proveedor (Compra, mantenimiento, arrendamiento).

Los activos deben poder asignarse a colaboradores y posteriormente devolverse.

La solución debe garantizar que un mismo activo no pueda quedar asignado a dos colaboradores al mismo tiempo.

FUNCIONES OBLIGATORIAS

1. Autenticación

La API deberá:

- Validar usuario y contraseña.
- Guardar contraseñas mediante hash seguro.
- Generar un JWT al autenticar correctamente.
- Validar expiración del token.
- Proteger los endpoints privados.
- Considerar al menos dos perfiles (Administrador, Operador).
- No se deberán almacenar ni comparar contraseñas en texto plano.

2. Administración de activos

La API deberá tener endpoints para registrar, consultar y actualizar activos.

Cada activo deberá contemplar al menos los siguientes datos:

- Id
- AssetCode
- SerialNumber
- Category
- Brand
- Model
- OwnershipType
- SupplierId
- Status
- CurrentLocation
- PurchaseDate
- RentalEndDate
- CreatedAt
- UpdatedAt

Reglas mínimas:

- AssetCode debe ser único.
- SerialNumber debe ser único cuando aplique.
- Un activo arrendado debe tener proveedor asociado.
- Los estados mínimos serían: Disponible, Asignado, Mantenimiento o Retirado.
- Un activo en estado de Asignado, mantenimiento o Retirado no puede asignarse a un colaborador.
- Un activo dado de baja no puede volver a asignarse.

3. Administración de colaboradores

Implementar al menos consulta y alta de colaboradores.

Reglas mínimas:

- Un colaborador inactivo no puede recibir activos.
- El correo electrónico debe tener formato válido.
- El número de empleado debe ser único.

4. Administración de proveedores

Implementar al menos consulta y alta de proveedores.

5. Asignación de activos

Implementar el proceso para asignar un activo a un colaborador.

Reglas obligatorias:

La solución deberá validar que:

- El activo exista.
- El activo esté disponible.
- El colaborador exista.
- El colaborador esté activo.
- El activo no tenga otra asignación activa.
- La asignación actualice el estado del activo a Asignado.
- Se cree un registro de auditoría o movimiento.
- Se registre el usuario que realizó la acción.
- No se pueda asignar el mismo activo dos veces.

6. Devolución de activos

Implementar el proceso de devolución de un activo asignado.

La devolución deberá:

- Validar que exista una asignación activa.
- Cerrar la asignación.
- Registrar fecha y condición de devolución.
- Cambiar el estado del activo a Disponible.
- Crear un movimiento de auditoría.
- Registrar el usuario que realizó la devolución.

7. Historial y trazabilidad

Implementar consultas que permitan conocer el historial de un activo.

El historial debe permitir identificar, como mínimo:

- Alta del activo.
- Asignación.
- Devolución.
- Cambio de estado.
- Cambio de ubicación.
- Usuario que realizó la acción.
- Fecha y hora.
- Observaciones.

8. Consulta de activos con filtros y paginación

El endpoint de consulta de activos deberá permitir filtros.

Ejemplo:

GET /api/assets?search=DELL&status=Disponible&category=Laptop

Filtros sugeridos:

- search
- status
- category

REGLA CRÍTICAS

1. La solución debe evitar que un mismo activo pueda ser asignado a dos colaboradores al mismo tiempo.

Ejemplo de escenario:

El usuario A intenta asignar la laptop TI-0001001. Casi al mismo tiempo, el usuario B intenta asignar la misma laptop. Solamente una asignación puede completarse la otra solicitud debe recibir una respuesta controlada.

Se evaluará especialmente el uso de:

- Stored Procedures.
 - Restricciones o índices únicos.
 - Manejo de concurrencia.
 - Errores de negocio controlados.
2. La solución deberá utilizar SQL Server y ADO.NET para la integración de datos. No se requiere Entity Framework Core. Las operaciones principales deberán utilizar Stored Procedures.

3. En temas de seguridad y uso de middleware, la API deberá implementar un mecanismo global de manejo de errores y no exponer ningún error como detalles internos de sql, datos secretos, información sensible o tokens.
4. El candidato deberá implementar una política razonable para limitar intentos de autenticación.
5. Todas las validaciones deberán realizarse del lado servidor como campos obligatorios, longitudes, formatos válidos, estados permitidos, reglas del negocio, manejo de errores claros etc.
6. La estructura del proyecto queda a criterio del candidato, pero se evaluará la lógica del negocio, modelos, acceso a datos, base de datos y pruebas.
7. La implementación de frontend es opcional. En caso de incluirlo, deberá contener al menos:
 - a. Inicio de sesión.
 - b. Listado de activos.
 - c. Alta de activo.
 - d. Asignación de activo.
 - e. Devolución de activo.
 - f. Consulta de historial.

ENTREGABLES

Deberás compartir un repositorio que incluya:

- Código fuente.
- Scripts de base de datos.
- Stored Procedures.
- README con pasos de instalación.
- Variables de entorno requeridas.
- Usuarios de prueba.
- Colección de Postman o archivo equivalente, opcional pero recomendado.
- Pruebas automatizadas.
- Explicación de decisiones técnicas.
- Tiempo aproximado invertido.
- Funcionalidades pendientes, en caso de existir.

El uso de herramientas de inteligencia artificial está permitido, sin embargo deberás indicar:

- Qué herramienta se utilizó.
- En qué parte del desarrollo se utilizó.
- Qué validaciones realizaste sobre el código generado o sugerido.
- Qué decisiones técnicas tomaste personalmente.

Durante la revisión técnica, deberás poder explicar y modificar su solución.

¡Mucho éxito!